

CSCE 606 SOFTWARE ENGINEERING · MODULE 04 · LECTURE NOTES

Architecture

The decisions that are expensive to reverse.

MODULE OUTCOMES

- Explain what distinguishes architecture from design.
- Identify the quality attributes that drive architectural choice.
- Compare layered, hexagonal, and event-driven styles honestly.
- Explain the monolith/microservice trade without slogans.
- Defer decisions that can be deferred.

1 What architecture is

THE USEFUL DEFINITION

Defining architecture as 'the high-level design' invites diagrams and settles nothing.

Architecture is the subset of design decisions that are expensive to change once made. Which datastore. Synchronous or asynchronous communication. One deployable unit or many. Where state lives. What consistency model users experience. Where the trust boundaries fall.

The practical value of this definition is that it tells you where to spend thought. A decision that can be reversed in an afternoon does not need a design review; a decision that would take six months to undo does.

Most decisions are not architectural and should be made as late as possible, by the person doing the work.

1.1 What drives it

Functional requirements rarely determine structure. Almost any architecture can be made to implement almost any feature; it is the **quality attributes** of Module 02 that constrain the shape.

Attribute	The question to ask	What it constrains
Scale	How much load, growing at what rate, in which dimension?	Partitioning, statelessness, caching strategy.
Latency	What must be fast, measured at which percentile?	Synchronous versus asynchronous; how many network hops are permitted on the critical path.

Attribute	The question to ask	What it constrains
Availability	What does an hour of downtime cost?	Redundancy, failover, how much of the system may share a failure domain.
Consistency	May a user see stale data, and for how long?	Whether distribution is viable at all for that data.
Security	Which boundaries must hold, against whom?	Process and network isolation, where secrets live (CSCE 611 Module 11).
Team structure	How many teams, and how independent must they be?	Deployment units. Conway's law is a constraint, not a suggestion.

2 Architectural styles

Style	Organising principle	Suits	Struggles with
Layered	Abstraction level; dependencies point one way.	Systems with a clear stack of concerns. Easy to explain and to enforce.	Changes that cut across layers — adding one field can require touching every layer, which is shotgun surgery by design.
Hexagonal / ports and adapters	A dependency-free core surrounded by pluggable adapters.	Systems with substantial domain logic that must outlive their frameworks and be testable in isolation.	Simple CRUD applications, where it is ceremony without payoff.
Event-driven	Components communicate by publishing and consuming events.	Decoupling producers from consumers; asynchronous work; natural audit logs; adding consumers without touching producers.	Debugging. There is no call stack. Understanding why something happened means reconstructing a causal chain across components, and ordering and duplicate handling become your problem.
Pipeline	Stages transforming a stream.	Data processing, compilers, media.	Interactive systems and anything needing to go backwards.

HEXAGONAL ARCHITECTURE, AND WHAT IT ACTUALLY BUYS

Place the domain logic at the centre with **no dependencies on anything external** — no ORM, no web framework, no HTTP types, no database. The core defines *ports* (interfaces it needs); everything external is an *adapter* implementing one.

The payoff is testability. The core can be tested with no database, no network, no fixtures, and no framework test harness. Those tests run in milliseconds and keep working when the framework is replaced — which it will be.

The cost is indirection. Every external interaction goes through an interface and an implementation, which is real navigational overhead. For an application that is mostly forwarding HTTP requests to SQL, this is pure ceremony.

The judgement is whether there is enough genuine domain logic to be worth protecting. If the answer is 'the domain logic is INSERT', there is not.

3 Monoliths and microservices

	Monolith	Microservices
Deployment	One artefact. Simple.	Many artefacts, needing orchestration, service discovery, and version compatibility management.
Inter-component calls	Function calls. They do not fail.	Network calls, which fail , time out, and may have executed despite failing (CSCE 611 Module 12).
Consistency	Database transactions work.	Transactions do not span services . You get sagas, compensating actions, and eventual consistency.
Debugging	One stack trace.	Distributed tracing, correlation IDs, and log aggregation — all of which must be built before you need them.
Scaling	The whole application scales together.	Each service scales independently, which matters if one component is the bottleneck.
Team autonomy	All teams coordinate on one codebase and one release.	Teams deploy independently.
Technology choice	One stack.	Per-service choice — a genuine benefit and a real fragmentation risk.

MICROSERVICES SOLVE AN ORGANISATIONAL PROBLEM

Read the table honestly: nearly every technical row favours the monolith. Function calls are faster and cannot fail halfway. Transactions work. One stack trace explains the whole request.

The row that reliably favours microservices is **team autonomy**. Twelve teams sharing one deployment pipeline is a genuine and growing coordination cost: every release requires everyone's changes to be ready, and one team's bug blocks everyone.

That is a Conway's law argument, not a technical one. If you have one team, or three, you are paying the full cost of distributed systems to solve a coordination problem you do not have.

Reasonable default: start with a well-modularised monolith. Extract a service when a specific, identified pressure justifies it — a team that needs independent release cadence, or a component whose scaling profile differs sharply from the rest. Extracting one service for a reason is very different from decomposing everything on principle.

THE DISTRIBUTED MONOLITH

Split a system into services without first fixing the coupling, and the result combines the disadvantages of both.

The symptoms are unmistakable: services that must be deployed together in a particular order; a single feature requiring coordinated changes across five repositories; a shared database that every service writes to; transactions that span services and are implemented with hope.

You now have the monolith's coupling *plus* network latency, partial failure, and distributed debugging. This is strictly worse than either alternative.

The modular boundaries must exist before the service boundaries. A service boundary is a module boundary with a network in it, and putting a network inside a bad boundary makes it worse, not better. If you cannot draw clean module boundaries within a monolith, splitting into services will not create them — it will only make the mess harder to see.

4 Making and deferring decisions

- 1 **Is this actually architectural?** Estimate the cost of reversing it in a year. If it is low, this is not an architectural decision and should be made by whoever is doing the work, now, without a meeting.
- 2 **Which quality attribute drives it?** Name it, with a number. 'We need it to scale' is not a driver; 'ten thousand concurrent users by Q3, 99th-percentile latency under 300 ms' is.
- 3 **What are the realistic options, and what does each cost?** Including the option of doing nothing yet.
- 4 **What would change the answer?** Write it down explicitly. This converts the decision from something to be defended into something to be reviewed when the trigger fires.
- 5 **Record it** as an architecture decision record: context, decision, alternatives considered, consequences accepted. Ten minutes now; an avoided argument in a year.

DEFERRING IS A SKILL

Every decision made early is made with the least information you will ever have about the problem. The value of deferring is that the decision gets made with more.

So part of the work is structuring the system so that *more* decisions can be deferred. Keeping persistence behind an interface defers the database choice. Keeping transport concerns out of the domain defers REST versus gRPC. Keeping the deployment unit singular defers the microservice question indefinitely.

But do not confuse deferring with ignoring. A decision that is never consciously made still gets made — implicitly, by whichever library someone imported first on a Tuesday. That is the worst case: expensive to reverse, and nobody chose it or recorded why.

Defer explicitly, write down that you are deferring, and name the trigger that will force the decision.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Martin Fowler — Software Architecture Guide	Including 'Who needs an architect?' and the microservices material, which is unusually balanced for the topic.
Fowler — MonolithFirst and Microservice Premium	The argument for §3's default, from someone who advocates microservices where they fit.
Alistair Cockburn — Hexagonal Architecture	The original description. Shorter and clearer than most of what has been written about it since.
Michael Nygard — Release It!	Stability patterns and failure modes in production systems. The chapter on failure cascades is worth the book.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 List the architectural decisions in a system you have built. For each, estimate the cost of reversing it now. Identify which you made deliberately and which were made by default.
- 2 Write an ADR for one of them, retrospectively: context, options, decision, consequences, and what would change the answer.
- 3 Take a small application and restructure its core to have no dependency on its framework or database. Measure test suite runtime before and after.
- 4 Find an open-source project that migrated to or from microservices. Read their write-up and identify which of the table's rows actually drove it.
- 5 For a system you know, name the quality attribute that most constrains its architecture, and state it with a number.
- 6 Identify a decision in your current project that is being made implicitly rather than deliberately. Write it down and decide it.
- 7 Take a monolithic codebase and identify where you would cut a service, based on module boundaries and change-coupling data from version control history.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What is the useful definition of architecture, and why does it help?
- 2 Why do quality attributes drive architecture while functional requirements usually do not?
- 3 What does hexagonal architecture buy, what does it cost, and when is it not worth it?
- 4 Which column of the monolith/microservices comparison reliably favours microservices, and what kind of argument is that?
- 5 What is a distributed monolith and why is it worse than either alternative?
- 6 Name the five steps of an architectural decision, and say which is most often skipped.
- 7 What is the difference between deferring a decision and ignoring it?